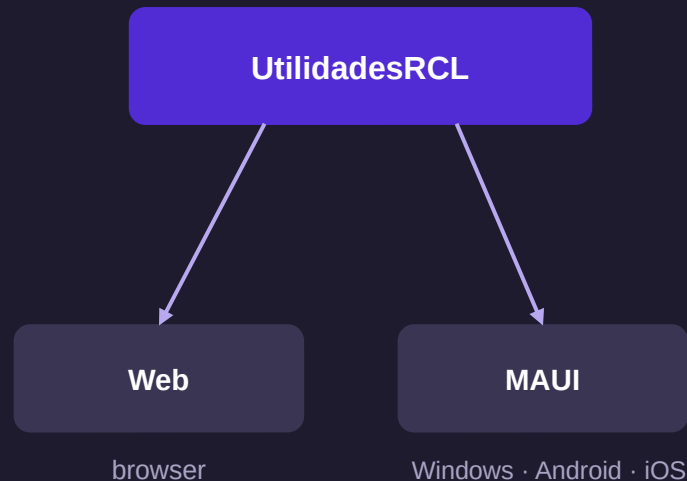


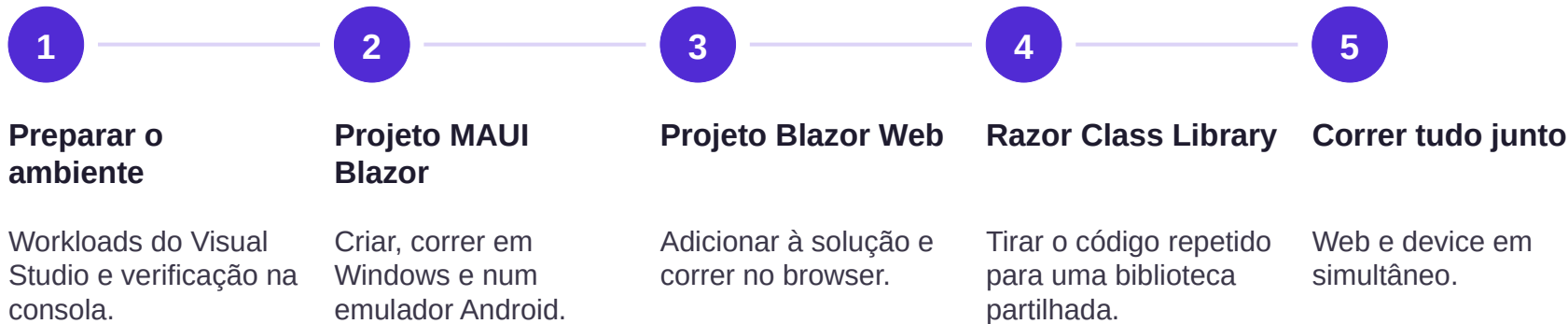
Ficha 2

App Utilidades

Uma solução, três projetos: o mesmo código Razor a correr no browser, no Windows e no Android.



Como lá vamos chegar



Nota: as etapas 1 a 3 são quase só configuração. Na etapa 4 entram os conceitos que acompanham o semestre inteiro: RCL, DTO e injeção de dependências.

Três projetos numa solução

A ficha divide-se em três partes, e cada uma cria um projeto. No fim, os três vivem na mesma solução.

1ª parte

Aplicação MAUI

`MauiUtilidades`

Uma app instalável, que corre em Windows e Android. É a janela nativa onde os nossos componentes aparecem.

2ª parte

Aplicação Web

`BlazorUtilidades`

A mesma ideia, mas no browser. Corre num servidor, que durante a aula é o próprio computador.

3ª parte

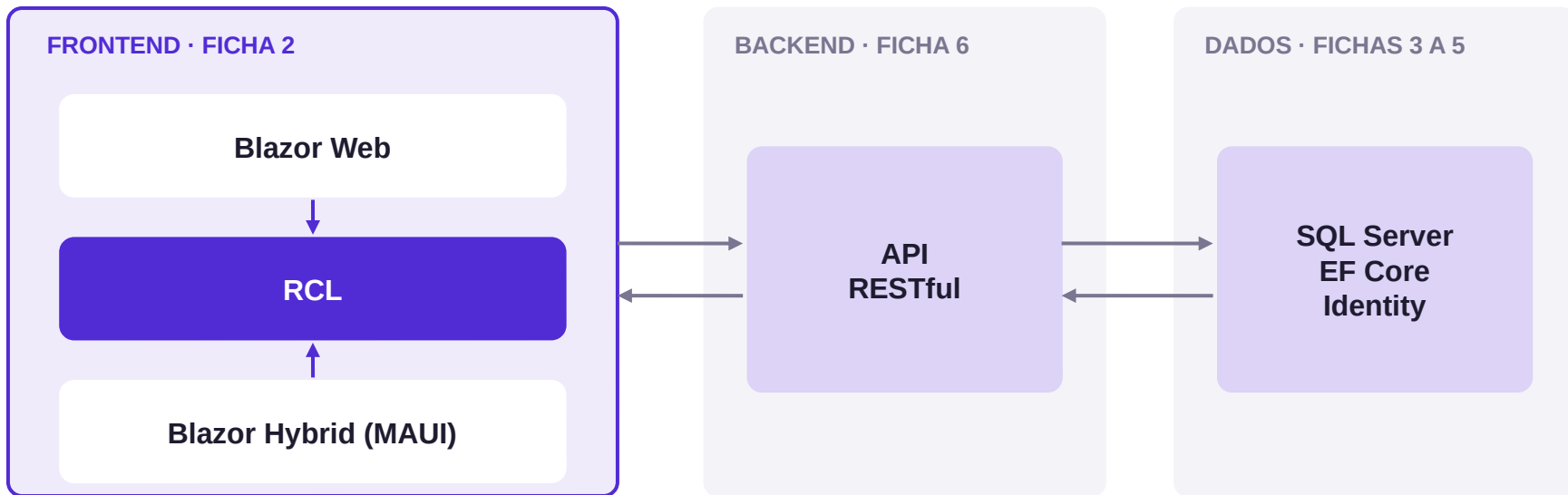
Razor Class Library

`UtilidadesRCL`

A biblioteca com o que as duas têm em comum: páginas, componentes, serviços e CSS.

Porquê a terceira? Sem ela, as duas primeiras teriam o mesmo código duplicado, e cada alteração teria de ser feita duas vezes.

Onde encaixa esta ficha no semestre



Esta é a arquitetura do Trabalho Prático. Hoje construímos só a parte da esquerda, com dados criados em memória. A API e a base de dados chegam nas fichas seguintes, e a forma como organizamos o código hoje é o que vai permitir ligá-las sem reescrever os componentes.

Blazor: quem faz o quê numa página

Contador.razor

```
@page "/contador"
@inject IJSRuntime JS

<h3 class="titulo">Contador</h3>
<p>Valor: @valor</p>
<button @onclick="Incrementar">+1</button>

@code {
    private int valor;
    private void Incrementar()
        => valor++;
}
```

Contador.razor.css

```
.titulo { color: #512BD4; }
```

HTML

Estrutura: títulos, parágrafos, botões, tabelas.

CSS

Aparência. Global (app.css, Bootstrap) ou isolado por componente.

C#

Estado, lógica e resposta aos eventos. É o papel que na Web clássica é do JavaScript.

Razor

A sintaxe com @ que junta HTML e C# no mesmo ficheiro.

O mesmo contador: JavaScript vs Blazor

HTML + JavaScript

```
<p>Valor: <span id="valor">0</span></p>
<button id="btn">+1</button>
```

```
<script>
  let valor = 0;
  document.getElementById("btn")
    .addEventListener("click", () => {
      valor++;
      document.getElementById("valor")
        .textContent = valor;
    });
</script>
```

Imperativo

Diz-se ao browser o que mudar, elemento a elemento.
Corre sempre no browser.

Blazor (Razor + C#)

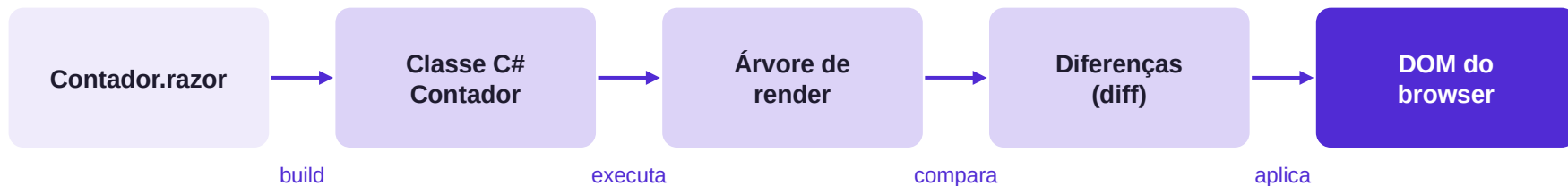
```
<p>Valor: @valor</p>
<button @onclick="Incrementar">+1</button>
```

```
@code {
  private int valor;
  void Incrementar() => valor++;
}
```

Declarativo

Descreve-se como a página depende do estado. O Blazor descobre o que mudou e atualiza.

O Razor é compilado para C#



Aproximadamente o que o compilador gera (obj/)

```
public partial class Contador : ComponentBase
{
    protected override void BuildRenderTree(
        RenderTreeBuilder builder)
    {
        builder.OpenElement(0, "p");
        builder.AddContent(1, "Valor: ");
        builder.AddContent(2, valor);
        builder.CloseElement();
        // ... botão, evento onclick ...
    }
}
```

Cada .razor vira uma classe C# normal. Quando o estado muda, o BuildRenderTree corre outra vez, o Blazor compara o resultado com o anterior e só envia ao browser o que mudou.

Em Server e Hybrid, o browser nunca vê C# nem Razor: só recebe HTML, CSS e o JavaScript do próprio Blazor.

CSS isolado e o JavaScript que ainda existe

CSS por componente

```
/* Contador.razor.css */  
.titulo { color: #512BD4; }  
  
/* o que o build gera */  
.titulo[b-3xxtam6d07] { color: #512BD4; }
```

O atributo b-... só existe nos elementos deste componente, por isso o estilo não se espalha para o resto da app. Todos os ficheiros .razor.css são juntos em {Projeto}.styles.css, que também inclui os da RCL.

JavaScript interop

```
@inject IJSRuntime JS  
  
<button @onclick="Avisar">Avisar</button>  
  
@code {  
    async Task Avisar() =>  
        await JS.InvokeVoidAsync("alert", "Olá!");  
}
```

Para o que só o browser sabe fazer (clipboard, localStorage, geolocalização) ou bibliotecas JS (mapas, gráficos). A RCL criada na ficha já traz um exemplo: [wwwroot/exampleJsInterop.js](#).

Um componente, três formas de o executar

Blazor Server

O C# corre no servidor. O browser recebe as alterações da página por uma ligação SignalR permanente.

É o que usamos no projeto Web desta ficha.

Blazor WebAssembly

O runtime .NET é descarregado e o C# corre dentro do próprio browser, sem servidor a processar os eventos.

Não usado nesta ficha.

Blazor Hybrid (MAUI)

O C# corre nativamente no dispositivo. Uma WebView embebida (BlazorWebView) só desenha o HTML.

É o que usamos no projeto MAUI.

A ideia a fixar: o componente .razor não sabe onde vai correr. Quem decide é o projeto anfitrião, e é isso que permite partilhá-lo na RCL.

Blazor Web App: o que é, afinal

É o template do .NET 8 para apps Blazor na Web. Por baixo é uma aplicação ASP.NET Core: um servidor que responde a pedidos HTTP e que usa componentes Razor para gerar as páginas.

Modelo de hospedagem

Onde corre o C#: no servidor (Server), no browser (WebAssembly) ou no dispositivo (Hybrid).

Render mode

Numa Blazor Web App, a escolha entre esses modos, feita por página ou para a app toda (Global).

```
BlazorUtilidades/Program.cs (simplificado)

var builder = WebApplication.CreateBuilder(args);

// 1. serviços (injeção de dependências)
builder.Services.AddRazorComponents()
    .AddInteractiveServerComponents();

var app = builder.Build();

// 2. pipeline: o que acontece a cada pedido HTTP
app.UseHttpsRedirection();
app.UseStaticFiles();
app.UseAntiforgery();

// 3. os componentes passam a ser páginas
app.MapRazorComponents<App>()
    .AddInteractiveServerRenderMode();

app.Run(); // arranca o servidor (Kestrel)
```

.NET MAUI e a BlazorWebView

O .NET MAUI (Multi-platform App UI) gera aplicações nativas para várias plataformas a partir de um único projeto. No template Blazor MAUI, a janela nativa contém uma BlazorWebView que mostra os nossos componentes.

- **Platforms/** código específico de cada SO (Android, iOS, Windows...).
- **MauiProgram.cs** arranque da app e registo de serviços.
- **wwwroot/index.html** página que a WebView carrega.
- **MainPage.xaml** página nativa que aloja a BlazorWebView.

Componentes Razor (.razor)

BlazorWebView (HTML + CSS)

MainPage.xaml · app nativa MAUI

Windows

Android

iOS

macOS

O C# corre no dispositivo, com acesso às APIs nativas (sensores, ficheiros, notificações).

Workloads necessários no Visual Studio

No Visual Studio Installer, em Modificar, confirmar que estão selecionados:

ASP.NET e desenvolvimento Web

Templates Blazor Web App e Razor Class Library.

Desenvolvimento .NET Multi-platform App UI

Templates MAUI, Android SDK e emuladores.

Processamento e armazenamentos de dados

SQL Server. Só necessário a partir da ficha 3.

Depois de instalar, ver se há atualizações pendentes.

PowerShell, Consola do Package Manager ou Terminal do VS

```
# ver o que está instalado  
dotnet workload list
```

```
# instalar o que falta  
dotnet workload install maui
```

```
# ou, dentro da pasta do projeto,  
# instalar tudo o que ele precisa  
dotnet workload restore
```

Criar o projeto Blazor MAUI

1 Novo projeto

Filtrar por Blazor Hybrid e escolher o primeiro template da lista.

2 Nome e local

MauiUtilidades. Este é o primeiro projeto da solução.

3 Framework

.NET 8 (Suporte de Longo Prazo), como nas figuras da ficha.

4 Correr

Escolher Windows Machine na barra de execução e carregar em F5.

Atenção ao nome do template

A ficha chama-lhe “Aplicativo Blazor .NET MAUI”. Nas versões recentes chama-se “Aplicativo .NET MAUI Blazor Hybrid”: é o mesmo template.

O segundo da lista, “Aplicativo Web e .NET MAUI Blazor Hybrid”, cria já os três projetos e não serve para esta ficha.

O que deve aparecer

Uma janela Windows com o menu lateral e as páginas Home, Counter e Weather.

É tudo código do template: ainda não há nada nosso no projeto. A primeira compilação demora alguns minutos.

Que ficheiros há num projeto destes

Extensão	O que é	Exemplos
.razor	componente Blazor: HTML e C# no mesmo ficheiro	Home.razor, NavMenu.razor
.razor.css	CSS isolado, que só afeta esse componente	MainLayout.razor.css
.cs	classe C# normal, ou o code-behind de outro ficheiro	MauiProgram.cs, Program.cs
.xaml	interface nativa do MAUI, escrita em XML	MainPage.xaml, App.xaml
.xaml.cs	o código da página XAML (a mesma classe, partida em dois)	MainPage.xaml.cs
.csproj	o projeto: framework de destino, pacotes, referências	MauiUtilidades.csproj
.json	configuração	launchSettings.json, appsettings.json

Convenção: um nome começado por `_` (como `_Imports.razor`) não é uma página; é um ficheiro que o compilador aplica a toda a pasta.

O projeto MAUI por dentro

```
MauiUtilidades/  
├─ Dependências/      pacotes NuGet e frameworks  
├─ Properties/        launchSettings.json  
├─ wwwroot/           index.html, css, favicon  
├─ Components/        Layout/, Pages/,  
                      Routes.razor, _Imports.razor  
├─ Platforms/         Android, iOS, MacCatalyst,  
                      Tizen, Windows  
├─ Resources/         ícones, splash, tipos de letra  
├─ App.xaml (+.cs)    arranque e recursos globais  
├─ MainPage.xaml (+.cs) aloja a BlazorWebView  
└─ MauiProgram.cs     configuração e serviços
```

Parte nativa

Platforms, Resources, App.xaml e MainPage.xaml. É a aplicação propriamente dita, com a janela e o acesso ao dispositivo.

Parte Blazor

wwwroot e Components. É o que a BlazorWebView mostra dentro da janela nativa, e é isto que vai passar para a RCL.

MainPage.xaml é a ponte: contém uma BlazorWebView que carrega o index.html e monta lá dentro o componente Routes.

Como corre em cada destino desta ficha

Destino	Runtime	Como o IL chega a código máquina
Blazor Web (servidor)	CoreCLR	JIT, no servidor
MAUI em Windows	CoreCLR	JIT, no PC
MAUI em Android	Mono	JIT em Debug; em Release, parte é compilada antes (AOT)
MAUI em iOS / macOS	Mono	tudo compilado antes (AOT): a Apple não permite JIT
Blazor WebAssembly	Mono em WASM	IL interpretado no browser (não usado nesta ficha)

```
MauiUtilidades.csproj
<TargetFrameworks>
  net8.0-android;net8.0-ios;net8.0-maccatalyst
</TargetFrameworks>
<!-- + net8.0-windows10.0.19041.0 no Windows -->
```

Um projeto MAUI é compilado uma vez por plataforma. No Android há ainda o empacotamento (APK) e a instalação no emulador: é por isso que o primeiro F5 demora tanto.

Emuladores Android

- **Abrir o Gestor de Dispositivos Android** (ícone na barra do VS ou Ferramentas > Android).
- **Criar os emuladores indicados na ficha** (pelo menos o Tablet, que é o usado no teste).
- **Arrancar primeiro o emulador**, esperar que o Android termine de iniciar e só depois recompilar e executar.
- **Escolher o emulador** na lista de destinos, no mesmo sítio onde estava Windows Machine.

Mac com Windows em máquina virtual

O Windows é Arm64 e o emulador precisa de uma imagem arm64-v8a. As imagens x86_64 que aparecem nas figuras da ficha não arrancam, e a virtualização aninhada ainda depende do Parallels, VMware ou UTM.

Quando não arranca

- Virtualização desligada na BIOS, ou Hyper-V / Windows Hypervisor Platform por ativar.
- Pouco espaço em disco: cada imagem ocupa vários GB.
- Primeiro deploy lento; não cancelar a meio.
- Emulador “preso”: fechar, e no Gestor fazer Cold Boot.

Alternativa

Telemóvel Android por USB, com a depuração USB ativa. Aparece na mesma lista de destinos.

Adicionar o projeto Blazor Web App

Clique direito na solução > Adicionar > Novo Projeto > “Blazor Web App”. Nome: BlazorUtilidades.

Opção	Valor	Porquê
Estrutura	.NET 8	Igual ao projeto MAUI.
Tipo de autenticação	Nenhum	Não precisamos de login nesta ficha.
Configurar para HTTPS	Sim	Por omissão, corre em https://localhost.
Interactive render mode	Server	O C# dos eventos corre no servidor.
Interactivity location	Global	Toda a app é interativa. Explicado mais à frente.
Include sample pages	Sim	Dá-nos Home, Counter e Weather para comparar.

O projeto Blazor Web por dentro

```
BlazorUtilidades/  
├─ Dependências/  
├─ Properties/  
│   └─ launchSettings.json    portas e perfis http/https  
├─ wwwroot/                  app.css, bootstrap, favicon  
├─ Components/  
│   ├── App.razor             documento HTML da página  
│   ├── Routes.razor          encaminhamento (router)  
│   ├── _Imports.razor        @using comuns  
│   ├── Layout/               MainLayout, NavMenu  
│   └─ Pages/                 Home, Counter, Weather  
├─ appsettings.json           configuração  
└─ Program.cs                 serviços e pipeline HTTP
```

Igual ao MAUI

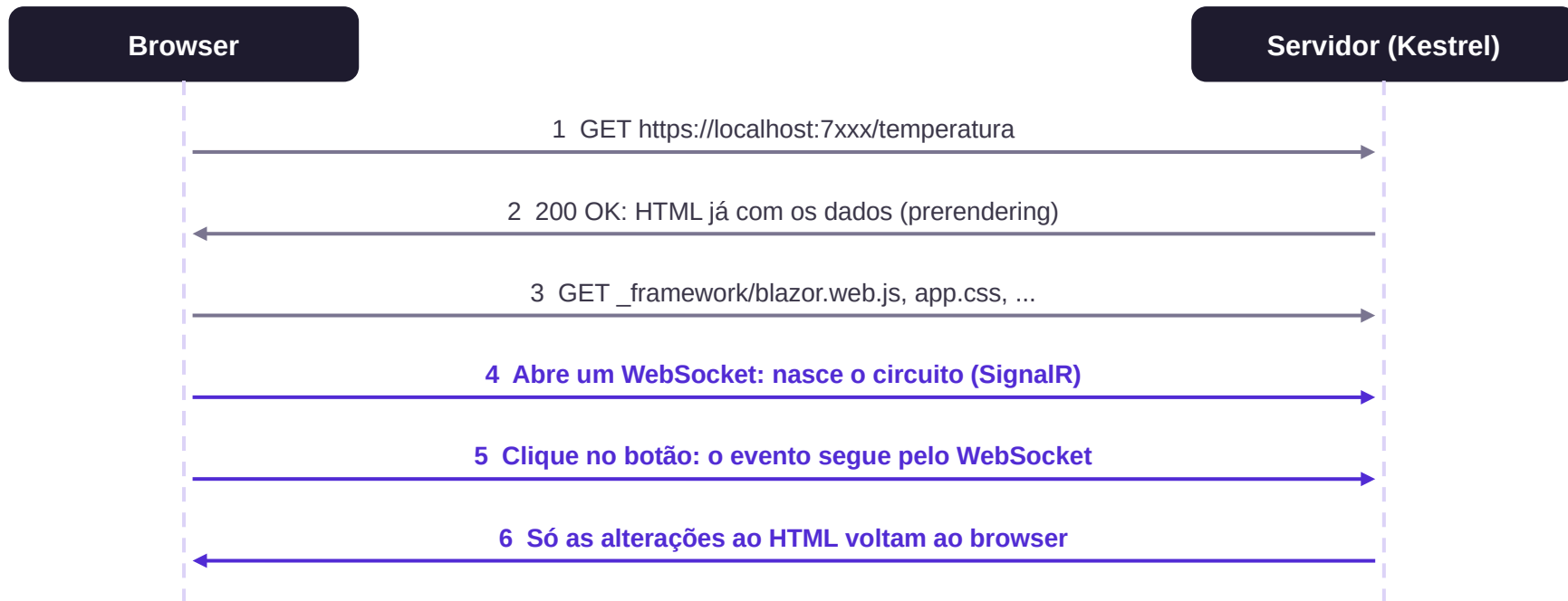
wwwroot, Components, Layout, Pages, Routes.razor e _Imports.razor. Os componentes são os mesmos.

Só existe aqui

Program.cs, App.razor, appsettings.json e launchSettings.json. São as peças de servidor Web, que a app MAUI não tem.

No MAUI o equivalente ao Program.cs é o MauiProgram.cs, e o papel do App.razor cabe ao wwwroot/index.html.

O que acontece quando se abre a página



cinzento: pedidos HTTP normais roxo: ligação SignalR, que fica aberta

Pedido HTTP vs circuito SignalR

	Pedido HTTP	Circuito SignalR (Blazor Server)
Ligação	abre, responde e fecha	fica aberta enquanto a página estiver aberta
Quem fala	só o browser pergunta	os dois lados enviam quando querem
Estado	nada fica guardado entre pedidos	o estado dos componentes vive no servidor
Custo	por pedido	memória no servidor por cada utilizador ligado

Consequências práticas

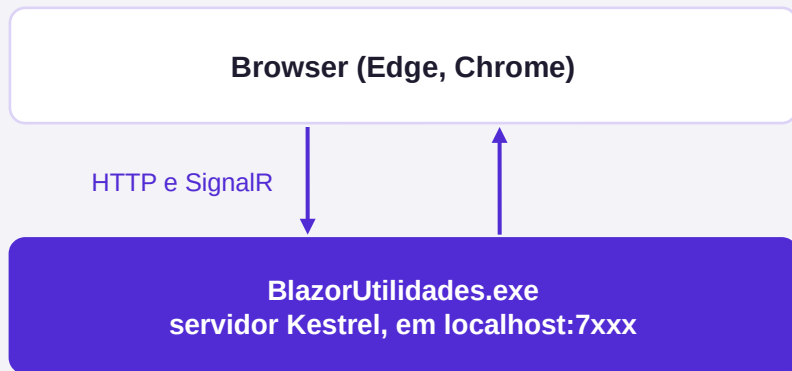
- Cada clique vai ao servidor e volta: com rede lenta, nota-se.
- Se a ligação cair, aparece “Attempting to reconnect” e o estado pode perder-se.
- O servidor guarda um circuito por utilizador. Voltamos a isto na injeção de dependências, porque muda o tempo de vida dos serviços.

E o HTTPS?

O perfil https usa um certificado de desenvolvimento. Na primeira execução o VS pede para confiar nele; se não pedir, `dotnet dev-certs https --trust`. As portas estão em `Properties/launchSettings.json`.

Onde está o servidor, afinal?

A tua máquina, durante o desenvolvimento



Ao carregar em F5, o Visual Studio arranca a aplicação e abre o browser. O servidor não está em lado nenhum especial: é um programa a correr no próprio computador.

Um projeto, não dois

Frontend e servidor são o mesmo projeto. O C# corre do lado do servidor e o browser recebe apenas HTML.

Em produção

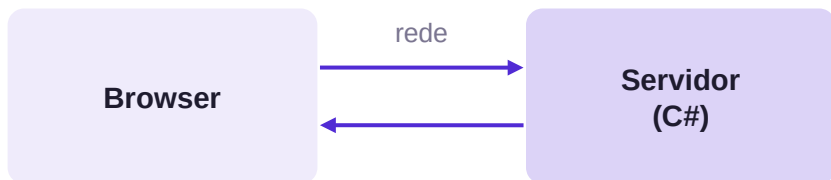
O mesmo projeto é publicado num servidor real (Windows, Linux ou Azure). Muda o sítio onde corre, não o código.

A partir da ficha 6

A API passa a ser um processo à parte: aí sim há dois programas a correr, e o frontend chama a API por HTTP.

E no MAUI? Não há HTTP nem servidor

Blazor Web (Server)



Blazor Hybrid (MAUI)



- **O index.html vem dentro da app.** Não há servidor, pedidos nem portas; funciona sem internet.
- **Os eventos passam da WebView para o C# localmente,** no mesmo processo, sem rede pelo meio.
- **O HTTP só aparece quando a app for buscar dados a uma API** (ficha 6), através de um HttpClient injetado num serviço.

O mesmo componente, com transportes completamente diferentes por baixo. É isto que a RCL aproveita.

Render modes e interatividade global

Static SSR

HTML gerado no servidor, sem eventos.

Interactive Server

Eventos no servidor, via SignalR. O nosso.

Interactive WebAssembly

Eventos no browser, com .NET em WebAssembly.

Interactive Auto

Começa em Server e muda para WebAssembly depois.

Porque Global?

Com Global, o render mode é definido uma única vez, no App.razor. Os componentes não têm @rendermode.

Isto é essencial para a etapa 4: o projeto MAUI não usa render modes, por isso os componentes partilhados têm de ser neutros.

```
BlazorUtilidades/Components/App.razor  
  
<Routes @rendermode="InteractiveServer" />
```

O que muda se não houver interatividade

A mesma página Counter, com e sem o render mode interativo:

	Static SSR (sem interatividade)	Interactive Server (o nosso)
Clicar no botão +1	não acontece nada: o C# do @onclick nunca chega a correr	o método corre e o número muda na hora
Atualizar o que se vê	só recarregando ou navegando para outra página	a página muda no sítio, sem recarregar
Escrever num campo	o valor só é conhecido quando o formulário for submetido	o C# sabe do valor à medida que se escreve
Ligação ao servidor	cada pedido abre e fecha	fica aberta enquanto a página estiver aberta

Static SSR não é um erro

Serve para páginas que são só para ler: um texto, uma listagem, um catálogo. É mais leve e não ocupa memória no servidor por cada utilizador.

Porque escolhemos Global

Assim todas as páginas são interativas e nenhum componente precisa de @rendermode. Isso é o que permite usá-los também no MAUI.

Porque é que os números mudam sozinhos?

Na app Web, o prerendering está ligado por omissão. O componente corre duas vezes:

1 No pedido HTTP

O servidor corre o componente para gerar o HTML inicial. A página aparece logo.

Com dados aleatórios, como os da página Weather do template ou do serviço de temperatura que vamos criar, vê-se a tabela a trocar de números logo depois de abrir. Não é um erro: é o componente a ser criado duas vezes.

2 Quando o circuito abre

O componente é criado de novo, agora interativo. O `OnInitializedAsync` corre outra vez.

Se quiserem desligar: `Components/App.razor`

```
<Routes @rendermode="new InteractiveServerRenderMode(
    prerender: false)" />
```

No MAUI isto não acontece: não há pedido HTTP inicial, logo não há prerendering.

Duas apps, o mesmo código duplicado

MauiUtilidades

MainLayout.razor

NavMenu.razor

Home · Counter · Weather

app.css · bootstrap

BlazorUtilidades

MainLayout.razor

NavMenu.razor

Home · Counter · Weather

app.css · bootstrap

A solução

Criar um terceiro projeto, uma Razor Class Library, com tudo o que é comum: componentes, páginas, serviços, CSS e imagens.

Os projetos MAUI e Web ficam “despidos”: só o arranque e o que é específico de cada plataforma.

Qualquer alteração tem de ser feita duas vezes.

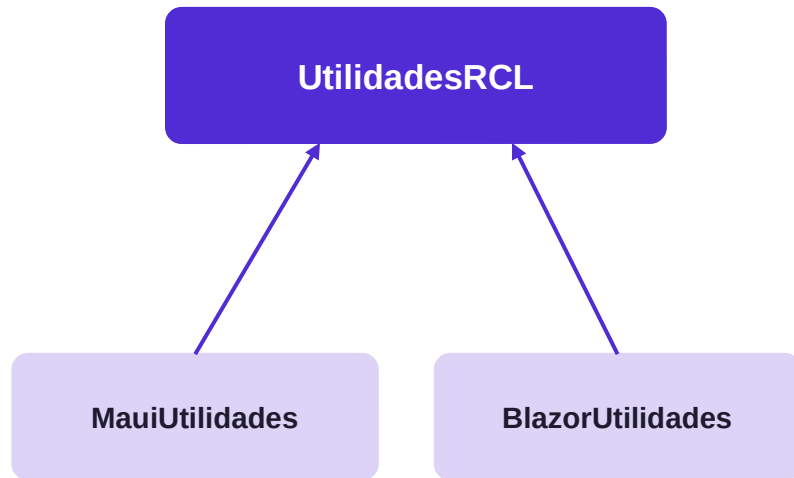
Razor Class Library (RCL)

Uma biblioteca .NET que, além de classes C#, pode conter componentes Razor e ficheiros estáticos. Compila para uma DLL que os outros projetos referenciam.

- Os componentes ficam disponíveis como qualquer outra classe, com `@using`.
- O conteúdo de `wwwroot` da RCL é servido num caminho próprio:

```
_content/UtilidadesRCL/app.css  
_content/UtilidadesRCL/bootstrap/...
```

Os links no `index.html` (MAUI) e no `App.razor` (Web) têm de passar a usar este caminho.



Referência de projeto: as setas vão das apps para a biblioteca. A RCL não conhece quem a usa.

O que fica em cada projeto

MauiUtilidades

- Platforms/ · Resources/
- wwwroot/index.html
- App.xaml · MainPage.xaml
- MauiProgram.cs
- Components/Layout/MainLayout
- Components/Pages/Home
- Routes.razor · _Imports.razor

BlazorUtilidades

- Properties/launchSettings.json
- Program.cs
- Components/App.razor
- Components/Layout/MainLayout
- Components/Layout/NavMenu
- Components/Pages/Home
- Routes.razor · _Imports.razor

Passa para a RCL

- Todo o wwwroot (menos index.html)
- NavMenu.razor do MAUI > Shared/
- Data/ DTO, Interfaces, Services
- Pages/ componentes da app
- _Imports.razor próprio

Estrutura da UtilidadesRCL

```
UtilidadesRCL/  
├── wwwroot/           bootstrap, app.css, imagens  
├── Data/  
│   ├── DTO/          Energia, Evento,  
│   │           Noticia, Temperatura  
│   ├── Interfaces/   IEnergiaService, ...  
│   └── Services/      EnergiaService, ...  
├── Pages/            HomeComponent,  
│           MenuComponent,  
│           EnergiaComponent, ...  
├── Shared/  
└── _Imports.razor    NavMenu.razor
```

DTO

Os dados que a app mostra e troca.

Interfaces

O contrato: o que o serviço promete.

Services

A implementação do contrato.

Pages

O que o utilizador vê.

DTO: os dados que viajam

Data Transfer Object: uma classe só com dados, sem lógica, feita para atravessar uma fronteira. O caso típico é a resposta de uma API ao cliente.

Entidade: espelha a tabela da base de dados

```
public class Produto
{
    public int Id { get; set; }
    public string Nome { get; set; }
    public decimal Preco { get; set; }
    public decimal PrecoCusto { get; set; }
    public int CategoriaId { get; set; }
    public Categoria Categoria { get; set; }
    public List<Encomenda> Encomendas { get; set; }
}
```

DTO: só o que o ecrã precisa

```
public class ProdutoDto
{
    public int Id { get; set; }
    public string Nome { get; set; } = "";
    public decimal Preco { get; set; }
    public string Categoria { get; set; } = "";
}
```

Esconde o que é interno

O PrecoCusto nunca deve chegar ao browser.

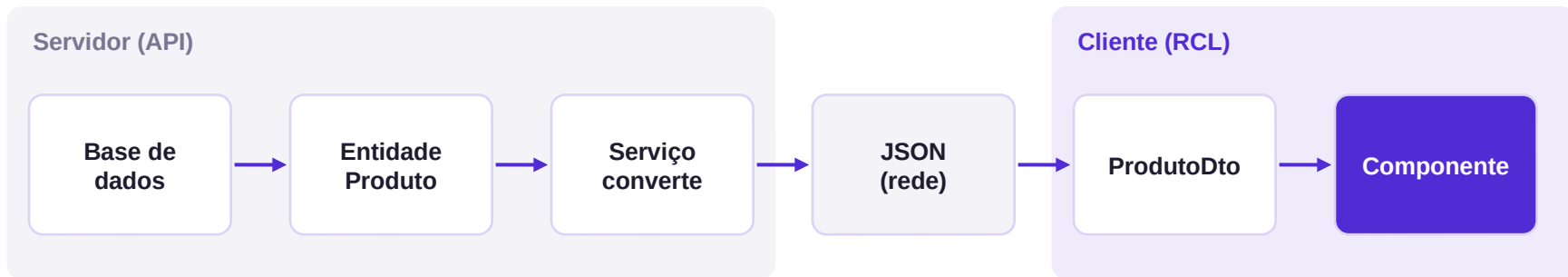
Evita ciclos e excesso

Produto > Categoria > Produtos...
rebenta a conversão para JSON.

Desacopla

A tabela pode mudar sem partir o frontend.

Onde entram os DTOs numa aplicação



E na ficha 2?

Não há base de dados nem API: o serviço cria os objetos em memória. Por isso a Temperatura é, por agora, só um modelo. Fica em Data/DTO porque, quando a API existir, é esta classe que vai receber o JSON.

Porque ficam na RCL?

A app Web e a app MAUI mostram os mesmos dados. Com os DTOs na RCL existe uma única definição, partilhada pelos dois frontends. O DTO passa a ser o contrato entre o frontend e a API.

O problema: o componente cria o serviço

Sem injeção de dependências

```
@code {  
    List<Temperatura>? previsao;  
  
    protected override async Task  
        OnInitializedAsync()  
    {  
        var servico = new TemperaturaService();  
        previsao = await  
            servico.ObterPrevisaoAsync(5);  
    }  
}
```

Funciona. Mas o componente passou a depender de uma classe concreta, e isso tem custos.

1

Mudar custa caro

Quando os dados vierem da API, é preciso alterar todos os componentes que fazem new TemperaturaService().

2

Dependências em cadeia

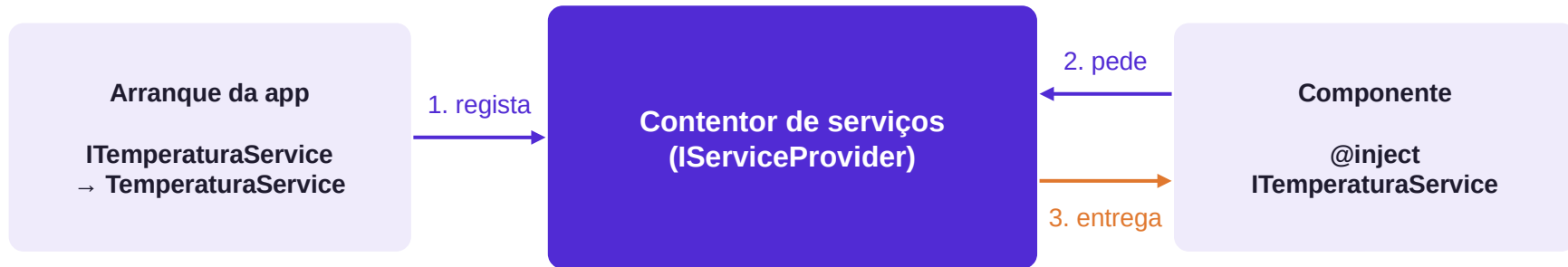
Se o serviço precisar de um HttpClient ou da BD, o componente também tem de saber criá-los.

3

Sem controlo do tempo de vida

Cada componente cria o seu objeto: não há forma de o partilhar nem de decidir quanto tempo vive.

A ideia: pedir em vez de criar



O componente diz o que precisa: uma interface, ou seja, um contrato. Não diz qual a classe nem como se constrói.

Quem decide a classe, cria o objeto e o entrega é o contentor, configurado uma única vez no arranque. O controlo de criar objetos passa do componente para a aplicação: é a inversão de controlo.

O contentor é uma lista de receitas

No arranque, a aplicação preenche uma lista: para cada contrato, qual a classe a usar e quanto tempo o objeto dura. Essa lista é o contentor.

Quando pedirem...	...o contentor cria	Duração
<code>ITemperaturaService</code>	<code>TemperaturaService</code>	Scoped
<code>IEnergiaService</code>	<code>EnergiaService</code>	Scoped
<code>NavigationManager</code>	(já vem do Blazor)	Scoped
<code>ILogger<T></code>	(já vem do .NET)	Singleton

Muitos serviços já lá estão sem ninguém os registar: vêm do próprio Blazor e do .NET. Os nossos juntam-se a essa lista.

Registado uma vez, usado em todo o lado

Qualquer componente ou serviço pode pedir o que está na lista, sem saber quem o criou.

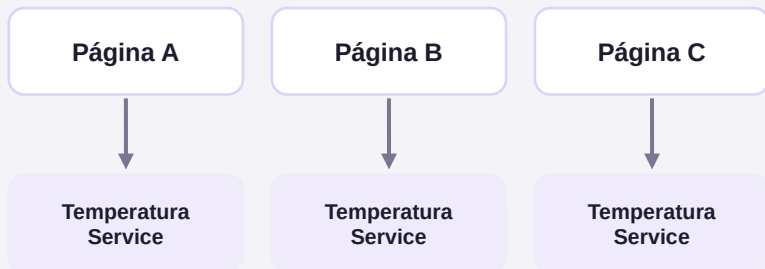
Quando um componente pede

1. O contentor procura o contrato na lista.
2. Vê se já existe um objeto válido para este âmbito.
3. Se não existir, cria-o, resolvendo primeiro o que o construtor dele pedir.
4. Entrega-o ao componente.

Se o contrato não estiver na lista, a aplicação falha ao abrir a página.

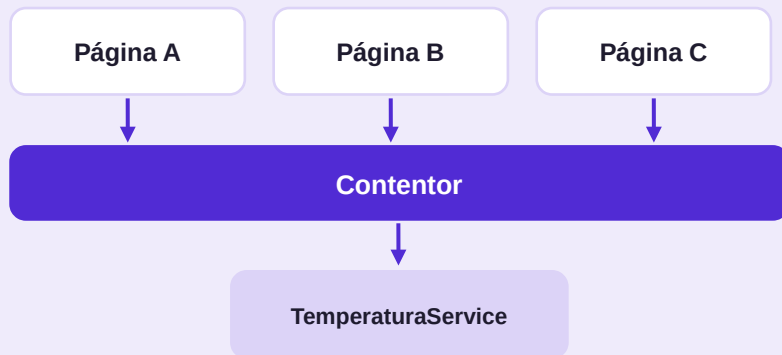
Com e sem injeção de dependências

Sem injeção: cada um constrói o seu



Três objetos diferentes. Para mudar de implementação, é preciso alterar as três páginas.

Com injeção: pedem ao contentor



Um objeto, criado e entregue pelo contentor. Mudar de implementação altera uma linha no arranque.

Trocar a implementação

Dados em memória hoje, API amanhã, sem tocar nas páginas.

Partilhar o mesmo objeto

Páginas diferentes podem trabalhar sobre os mesmos dados.

Decidir num sítio só

O arranque da app define o que se usa e quanto tempo dura.

Em .NET são três passos

1 Registrar

No arranque de cada app
(MauiProgram.cs e Program.cs):

```
builder.Services  
    .AddScoped<  
        ITemperaturaService,  
        TemperaturaService>();
```

2 Pedir

Num componente, com `@inject`. Numa classe, pelo construtor:

```
public class RelatorioService  
{  
    private readonly  
        ITemperaturaService _temp;  
  
    public RelatorioService(  
        ITemperaturaService temp)  
        => _temp = temp;  
}
```

3 Resolver

Ao criar um objeto, o contentor vê o construtor, cria primeiro o que lá está pedido e só depois o objeto. Se faltar um registo, dá erro logo aí.

*RelatorioService pede
ITemperaturaService, que é criado
primeiro e passado ao construtor.*

Já estão a usar DI: `AddRazorComponents()`, `AddMauiBlazorWebView()` e `@inject` `NavigationManager` nos templates. Na ficha 3, o `DbContext` da base de dados também entra assim.

Tempo de vida: quanto dura cada objeto

Registro	Uma instância por...	Usar para
AddSingleton	a app inteira (no servidor, compartilhada por todos)	configuração, caches
AddScoped	âmbito (scope), que depende do anfitrião: ver abaixo	serviços de dados, DbContext
AddTransient	cada vez que é pedido	objetos leves e sem estado

O que é o âmbito em cada caso

- **Blazor Server:** cada ligação (circuito) de um utilizador.
- **Blazor Hybrid:** a BlazorWebView, na prática a app toda.
- **Web API (ficha 6):** cada pedido HTTP.

Cuidado com o Singleton na Web

No Blazor Server a app corre no servidor, e um Singleton é o mesmo objeto para todos os utilizadores ligados. Guardar lá o carrinho de compras de alguém é um bug de privacidade. No MAUI não há esse problema: só há um utilizador.

A utilidade Temperatura (1/2)

Data/DTO/Temperatura.cs

```
namespace UtilidadesRCL.Data.DTO;  
  
public record Temperatura(  
    string Dia,  
    string TemperaturaC,  
    string Resumo);
```

Data/Interfaces/ITemperaturaService.cs

```
using UtilidadesRCL.Data.DTO;  
  
namespace UtilidadesRCL.Data.Interfaces;  
  
public interface ITemperaturaService  
{  
    Task<IEnumerable<Temperatura>>?  
        LoadTemperaturasAsync();  
}
```

Data/Services/TemperaturaService.cs

```
public class TemperaturaService : ITemperaturaService  
{  
    public async Task<IEnumerable<Temperatura>>  
        LoadTemperaturasAsync()  
    {  
        await Task.Delay(1000);  
  
        var temperaturas = new[]  
        {  
            new Temperatura("24-08-2026", "32", "Quente"),  
            new Temperatura("25-08-2026", "23", "Ameno"),  
            new Temperatura("26-08-2026", "20", "Frio"),  
            // ...  
        };  
        return temperaturas;  
    }  
}
```

Porquê o Task.Delay? Simula o tempo de espera de uma API, para se ver o “Carregando...” no ecrã.

A utilidade Temperatura (2/2)

```
Pages/TemperaturaComponent.razor (a tabela está simplificada)

@page "/temperaturas"

<h3>Temperaturas para esta semana</h3>

@if (temperaturas is null)
{
    <span>Carregando temperaturas...</span>
}
else
{
    <table>
        <tr><td>Dia</td><td>Temperatura</td><td>Resumo</td></tr>
        @foreach (var t in temperaturas)
        {
            <tr><td>@t.Dia</td><td>@t.TemperaturaC</td>
                <td>@t.Resumo</td></tr>
        }
    </table>
}

@code {
    private IEnumerable<Temperatura>? temperaturas;

    protected override async Task OnInitializedAsync()
    => temperaturas =
        await temperaturaService.LoadTemperaturasAsync();
}
```

Repare-se

- `@page` dá a rota; é o que o NavMenu usa.
- O teste `is null` mostra a mensagem de espera enquanto os dados não chegam.
- Não há `@rendermode`: o componente serve a Web e o MAUI.
- O `temperaturaService` vem do `_Imports` da RCL, onde estão os `@inject`.

Duas formas de escrever o mesmo componente

```
Tudo no .razor (TemperaturaComponent)

@page "/temperaturas"

<h3>Temperaturas</h3>
@* ... a tabela ... *@

@code {
    private IEnumerable<Temperatura>? temperaturas;

    protected override async Task OnInitializedAsync()
        => temperaturas =
            await
            temperaturaService.LoadTemperaturasAsync();
}
```

É a mesma classe

A palavra partial diz ao compilador que a classe está repartida por dois ficheiros. É o mesmo mecanismo do MainPage.xaml e MainPage.xaml.cs.

```
EnergiaComponent.razor

@page "/energias"

<h3>Preços da Energia</h3>
@* ... a tabela ... *@
```

```
EnergiaComponent.razor.cs (code-behind)

using UtilidadesRCL.Data.DTO;

namespace UtilidadesRCL.Pages;

public partial class EnergiaComponent
{
    private IEnumerable<Energia>? energias;

    protected override async Task OnInitializedAsync()
        => energias =
            await energiaService.LoadEnergiasAsync();
}
```

A solução de referência usa as duas formas. Com pouco código, o bloco @code chega; quando cresce, separar ajuda a ler.

Registrar os serviços nos dois anfitriões

A RCL é só uma biblioteca: não arranca sozinha e não tem contentor. Quem regista os serviços são as apps que a usam, cada uma no seu arranque.

As mesmas quatro linhas em MauiProgram.cs e em Program.cs

```
using UtilidadesRCL.Data.Interfaces;  
using UtilidadesRCL.Data.Services;  
  
// ...  
  
builder.Services.AddScoped<IEnergiaService, EnergiaService>();  
builder.Services.AddScoped<IEventoService, EventoService>();  
builder.Services.AddScoped<INoticiaService, NoticiaService>();  
builder.Services.AddScoped<ITemperaturaService,  
    TemperaturaService>();
```

Quando forem muitos serviços, vale a pena criar na RCL um método de extensão AddUtilidades() que os regista todos, e chamá-lo com uma linha em cada arranque.

Porquê nos dois

Cada app tem o seu contentor. Registrar só num lado faz a outra falhar ao abrir a página que usa o serviço.

O erro quando falta

“Cannot provide a value for property ...
There is no registered service of type
ITemperaturaService.”

O que ganhamos: trocar a implementação

Mais à frente: os dados vêm da API (ficha 6)

```
public class TemperaturaApiService : ITemperaturaService
{
    private readonly HttpClient _http;

    public TemperaturaApiService(HttpClient http)
        => _http = http;

    public async Task<IEnumerable<Temperatura>>
        LoadTemperaturasAsync()
        => await _http.GetFromJsonAsync<IEnumerable<Temperatura>>(
            "api/temperaturas") ?? [];
}
```

A única alteração no registo

```
// antes
AddScoped<ITemperaturaService,
    TemperaturaService>();

// depois
AddScoped<ITemperaturaService,
    TemperaturaApiService>();
```

É isto que junta tudo: interface, DI e DTO.

- O **TemperaturaComponent** não muda uma linha.
- O contentor vê que **TemperaturaApiService** precisa de um **HttpClient** e resolve-o primeiro.
- O JSON da API é convertido para **Temperatura**: aqui a classe já faz de DTO.

_Imports.razor: um diferente por projeto

Os `@using` de um `_Imports.razor` aplicam-se a todos os `.razor` da mesma pasta e subpastas. Como cada projeto usa coisas diferentes, o conteúdo muda.

UtilidadesRCL/_Imports.razor

```
@* ... usings do template ... *@
@using UtilidadesRCL
@using UtilidadesRCL.Pages
@using UtilidadesRCL.Data.DTO
@using UtilidadesRCL.Data.Interfaces

@inject IEnergiaService energiaService
@inject IEventoService eventoService
@inject INoticiaService noticiaService
@inject ITemperaturaService temperaturaService
```

O `@inject` aqui poupa repetição

Os quatro serviços ficam disponíveis em todos os componentes da RCL, sem cada um os declarar.

MauiUtilidades/Components/_Imports.razor

```
@* ... usings do template ... *@
@using MauiUtilidades.Components
@using UtilidadesRCL
@using UtilidadesRCL.Data.Interfaces
```

BlazorUtilidades/Components/_Imports.razor

```
@* ... usings do template ... *@
@using static
Microsoft.AspNetCore.Components.Web.RenderMode
@using BlazorUtilidades.Components
```

Cada app importa o que usa

O Home de cada projeto declara o seu próprio `@using UtilidadesRCL.Pages`, por usar o `HomeComponent`.

As páginas da RCL não aparecem?

O router não procura na RCL

No Routes.razor dos dois projetos:

```
AdditionalAssemblies="new[] {  
    typeof(UtilidadesRCL._Imports).Assembly }"
```

O servidor Web não conhece as rotas

No Program.cs do BlazorUtilidades:

```
app.MapRazorComponents<App>()  
    .AddInteractiveServerRenderMode()  
    .AddAdditionalAssemblies(  
        typeof(UtilidadesRCL._Imports).Assembly);
```

Outros sintomas

- **Página sem estilos:** links do CSS ainda sem o prefixo `_content/UtilidadesRCL/`.
- **"Cannot provide a value":** serviço não registado num dos arranques.
- **Tipo não encontrado:** falta referência de projeto ou `@using` no `_Imports`.

Correr a Web e um device ao mesmo tempo

1

Compilar a solução inteira e corrigir erros.

2

Correr BlazorUtilidades (perfil https) e confirmar no browser.

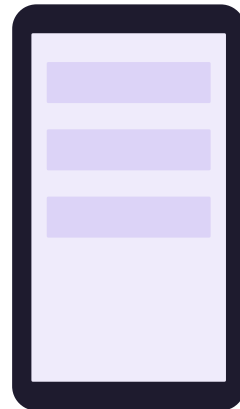
3

Correr MauiUtilidades em Windows Machine e no emulador Android.

4

Clique direito na solução > Configurar Projetos de Inicialização > Vários projetos de inicialização > Iniciar nos dois.

localhost:7xxx/temperatura



O mesmo TemperaturaComponent, dois anfitriões diferentes.

No arranque múltiplo, o device usado é o que estava selecionado antes de abrir a configuração.

ANEXO

Como isto funciona por baixo

Compilação, SDK e a relação entre .NET, ASP.NET Core e MAUI. Não é preciso para executar a ficha, mas responde às perguntas que costumam aparecer.

Do código C# às instruções da máquina

1. O que escrevemos (C#)

```
static int Soma(  
    int a, int b)  
=> a + b;
```

compilador
(SDK)

2. O que fica na DLL (IL)

```
ldarg.0  
ldarg.1  
add  
ret
```

JIT
(runtime)

3. O que o CPU executa (x64)

```
lea eax, [rcx+rdx]  
ret
```

ao compilar: dotnet build ou F5 no Visual Studio

ao executar: cada método é traduzido na primeira vez que é chamado

IL: Intermediate Language

Instruções para uma máquina virtual, iguais em qualquer sistema. A DLL guarda o IL e a descrição das classes.

Porquê dois passos?

A mesma DLL corre em Windows, Linux ou macOS, em x64 ou ARM. Quem a adapta ao processador é o runtime.

O que acontece quando se carrega em F5

1

Restore

O MSBuild lê o .csproj e vai buscar os pacotes NuGet indicados.

2

Geração de código

Cada .razor é transformado em C# por um gerador, antes de haver compilação.

3

Compilação

O compilador C# (Roslyn) junta o código escrito e o gerado e produz um assembly com IL.

4

Empacotamento

Ícones, CSS e recursos são copiados. No Android, é ainda construído o APK.

5

Execução

O runtime carrega o assembly e o JIT traduz cada método na primeira chamada.

As pastas obj e bin são geradas: podem ser apagadas e não vão para o Git.

```
MauiUtilidades/  
├── obj/                trabalho intermédio  
│   ├── Debug/net8.0/  
│   │   ├── generated/  o C# gerado a partir  
│   │   │               dos .razor  
│   │   ├── *.dll        resultado da compilação  
│   └── bin/            resultado final  
│       ├── Debug/net8.0/  
│       │   ├── MauiUtilidades.dll  o nosso IL  
│       │   ├── *.dll               pacotes usados  
│       │   └── *.pdb               símbolos de debug
```

Ver o IL de uma DLL

ILSpy, ou sharplab.io para ver C#, IL e assembly a par.

SDK, runtime e workloads

SDK: para desenvolver

Compilador C#, comando dotnet, MSBuild, templates, NuGet.

Runtime: para executar

CLR
JIT, garbage collector

Bibliotecas base
List, LINQ, HttpClient

Workloads: extensões opcionais do SDK

maui-android

maui-ios

maui-windows

wasm-tools

O que tenho instalado?

```
dotnet --list-sdks  
dotnet --list-runtimes  
dotnet workload list
```

global.json, na pasta da solução

```
{  
  "sdk": {  
    "version": "8.0.400",  
    "rollForward": "latestFeature"  
  }  
}
```

Com vários SDKs instalados, o global.json fixa qual a solução usa. Evita o clássico “no meu PC compila”.

.NET, ASP.NET Core, Blazor e MAUI

A nossa aplicação

ASP.NET Core
web e servidor

.NET MAUI
apps nativas

EF Core
base de dados

Plataforma .NET 8
runtime (CLR), bibliotecas base, compilador C#

Windows · Linux · macOS · Android · iOS

Cada camada assenta na de baixo. ASP.NET Core, MAUI e EF Core são bibliotecas, não plataformas separadas: todas correm sobre o mesmo runtime e escrevem-se na mesma linguagem.

ASP.NET Core

As bibliotecas para fazer servidores e interfaces web em .NET: Kestrel, middleware, Web API, SignalR e Blazor.

Blazor

Faz parte do ASP.NET Core. No projeto MAUI, os componentes vêm de lá, através do pacote Components.WebView.Maui.

Nomes parecidos, coisas diferentes

.NET Framework 4.8 só corre em Windows e é a geração antiga. ASP.NET sem “Core” é o mesmo caso.

ANEXO

A refatoração, passo a passo

A sequência exata da etapa 4, para acompanhar durante a aula ou para consultar quando alguma coisa não compilar.

Criar a biblioteca e ligá-la aos projetos

1

Criar o projeto UtilidadesRCL

- Clique direito na solução > Adicionar > Novo Projeto.
- Procurar “Biblioteca de Classes Razor”.
- Nome UtilidadesRCL, na pasta da solução.
- Framework .NET 8. Deixar Support pages and views por marcar.

O que o template traz

Component1.razor e ExampleJsInterop.cs, que podem ser apagados, e no wwwroot o background.png e o exampleJsInterop.js, que a figura 29 mantém.

O .csproj já traz o pacote Microsoft.AspNetCore.Components.Web: é ele que permite ter componentes numa biblioteca.

2

Adicionar as referências de projeto

- No MauiUtilidades: clique direito em Dependências > Adicionar Referência de Projeto > marcar UtilidadesRCL.
- Repetir no BlazorUtilidades.
- Confirmar: em Dependências > Projetos passa a aparecer UtilidadesRCL.

As setas vão num só sentido

As duas apps conhecem a biblioteca. A biblioteca não conhece as apps, e nunca deve ter referência para elas.

Pastas da RCL e conteúdo do wwwroot

3

Criar as pastas na RCL

```
UtilidadesRCL/  
├── Data/  
│   ├── DTO/  
│   ├── Interfaces/  
│   └── Services/  
├── Pages/  
└── Shared/
```

Clique direito no projeto > Adicionar > Nova Pasta, uma de cada vez.

Arrastar move, não copia

Se preferir, use Cortar e Colar. Confirme sempre que o ficheiro deixou de existir na origem, senão ficam duas versões do mesmo CSS.

4

Mover o conteúdo dos wwwroot

- Arrastar do BlazorUtilidades para o wwwroot da RCL: a pasta bootstrap, o app.css e o favicon.png.
- No MauiUtilidades, apagar a pasta css e o favicon: são duplicados dos que já foram.
- No MauiUtilidades fica apenas o index.html.
- No BlazorUtilidades o wwwroot fica vazio e pode ser apagado.

Porquê o index.html fica

É o documento que a BlazorWebView carrega.
É específico do MAUI.

Mover o NavMenu e limpar o template

5

NavMenu do MAUI para a RCL

- Arrastar MauiUtilidades/Components/Layout/NavMenu.razor para Shared, na RCL.
- Levar também o NavMenu.razor.css, se existir.
- O NavMenu do BlazorUtilidades fica onde está: é o que a ficha pede.

Porque é que só um deles se move

Os menus das duas apps vão ter entradas iguais, mas o da Web fica separado para poder ter itens próprios mais tarde. Partilhado, na RCL, fica o do MAUI.

6

Apagar o que o template trouxe

- Apagar Counter.razor e Weather.razor nos dois projetos.
- Home.razor fica em ambos, como mostra a figura 28.
- Nos dois NavMenu, apagar os NavLink que apontavam para counter e weather, senão ficam ligações para páginas que já não existem.

Compilar agora

Vale a pena compilar antes de mexer nos caminhos: assim sabe-se que o que falhar a seguir vem dos passos seguintes.

Corrigir os caminhos dos ficheiros estáticos

Tudo o que vem de uma RCL é servido sob `_content/NomeDaRCL/`. Como o CSS mudou de projeto, os caminhos têm de mudar.

7

Antes

```
MauiUtilidades/wwwroot/index.html

<link rel="stylesheet"
      href="css/bootstrap/bootstrap.min.css" />
<link rel="stylesheet" href="css/app.css" />
<link rel="icon" href="favicon.png" />
```

O que não se altera

A linha do `MauiUtilidades.styles.css` (ou `BlazorUtilidades.styles.css`). É gerada pelo build e já inclui o CSS isolado dos componentes da RCL.

Depois

```
e o mesmo em BlazorUtilidades/Components/App.razor

<link rel="stylesheet" href=
  "_content/UtilidadesRCL/bootstrap/bootstrap.min.css" />
<link rel="stylesheet"
  href="_content/UtilidadesRCL/app.css" />
<link rel="icon"
  href="_content/UtilidadesRCL/favicon.png" />
```

Como confirmar

Correr a app Web e abrir o F12, separador Rede. Um 404 num ficheiro `.css` indica caminho errado: a página aparece sem o Bootstrap, com os tipos de letra e os botões do browser.

_Imports.razor: dizer onde estão os componentes

8

Ao mudarem de projeto, os componentes mudaram de namespace. Cada projeto precisa de saber onde procurá-los.

UtilidadesRCL/_Imports.razor

```
@using System.Net.Http
@using Microsoft.AspNetCore.Components.Routing
@using Microsoft.AspNetCore.Components.Web
@using UtilidadesRCL.Data.DTO
@using UtilidadesRCL.Data.Interfaces
```

Porque é que o MAUI precisa de mais um

O MainLayout do MAUI usa o NavMenu, que agora vive em UtilidadesRCL.Shared. Sem esse @using, o compilador diz que não encontra o componente NavMenu.

MauiUtilidades/Components/_Imports.razor

```
@* ... as linhas que já lá estavam ... *@
@using UtilidadesRCL.Pages
@using UtilidadesRCL.Shared
```

BlazorUtilidades/Components/_Imports.razor

```
@* ... as linhas que já lá estavam ... *@
@using UtilidadesRCL.Pages
```

O _Imports.razor aplica-se a todos os .razor da sua pasta e subpastas.

Dizer ao router que há páginas na RCL

9

Por omissão, o router só procura páginas no assembly da própria app. As páginas que estão na biblioteca têm de ser declaradas.

```
Routes.razor dos dois projetos (no MAUI: typeof(MauiProgram))  
  
@using UtilidadesRCL.Pages  
  
<Router AppAssembly="@typeof(Program).Assembly"  
        AdditionalAssemblies="new[] {  
            typeof(HomeComponent).Assembly }">
```

```
Program.cs, só no BlazorUtilidades  
  
app.MapRazorComponents<App>()  
    .AddInteractiveServerRenderMode()  
    .AddAdditionalAssemblies(  
        typeof(UtilidadesRCL._Imports).Assembly);
```

Na app Web são precisas as duas alterações: o router encaminha, e o servidor tem de conhecer as rotas.

Sintoma quando falta

A aplicação compila e arranca, mas ao abrir uma página da RCL aparece "Sorry, there's nothing at this address".

Porquê `_Imports`

Serve só para apontar para o assembly da RCL. Qualquer tipo dela servia; este existe sempre.

Confirmar que ficou tudo bem

10

A lista de verificação

- A solução compila sem erros nem avisos de tipos em falta.
- A app Web abre e o Home aparece com estilos e com o menu lateral.
- A app MAUI abre em Windows Machine, também com estilos.
- Os dois projetos têm a referência a UtilidadesRCL em Dependências.
- O wwwroot da RCL tem o bootstrap, o app.css e o favicon.

Só depois disto vale a pena passar aos DTO, serviços e componentes das utilidades.

Se alguma coisa falhar

Página sem formatação

caminho `_content` errado no `index.html` ou no `App.razor`

“Nothing at this address”

falta `AdditionalAssemblies` ou `AddAdditionalAssemblies`

NavMenu não encontrado

falta `@using UtilidadesRCL.Shared` no `_Imports` do MAUI

Tipo não encontrado

falta a referência de projeto

Criar os dados: DTO, interfaces e serviços

Alínea d da ficha. Para cada utilidade (Energia, Evento, Noticia, Temperatura) fazem-se três ficheiros, sempre pelo mesmo molde.

Data/DTO/

`Temperatura.cs`

O formato dos dados que vão ser mostrados. Um record posicional chega: só guarda valores.

Data/Interfaces/

`ITemperaturaService.cs`

O contrato: promete devolver uma coleção de Temperatura, de forma assíncrona.

Data/Services/

`TemperaturaService.cs`

A implementação: hoje devolve dados fixos em memória; amanhã irá buscá-los à API.

Porquê três ficheiros para uma coisa tão simples

Porque separam o que muda do que fica. O componente só conhece o DTO e a interface; a classe que vai buscar os dados pode ser trocada sem que ele saiba. É o mesmo molde que vai ser usado no trabalho prático, aí com uma base de dados e uma API por trás.

Criar os componentes em Pages

Alínea e da ficha. São seis, e não têm todos o mesmo papel.

Componente	Rota	O que faz
<code>EnergiaComponent</code>	<code>/energias</code>	pede os dados ao serviço e mostra-os numa tabela
<code>EventoComponent</code>	<code>/eventos</code>	igual, com os campos dos eventos
<code>NoticiaComponent</code>	<code>/noticias</code>	igual, com os campos das notícias
<code>TemperaturaComponent</code>	<code>/temperaturas</code>	igual, com os campos da temperatura
<code>HomeComponent</code>	(sem rota)	recebe um parâmetro <code>MensagemBody</code> e é usado pelo Home de cada app
<code>MenuComponent</code>	(sem rota)	variante do menu, usada como alternativa ao <code>NavMenu</code>

Os quatro com rota são iguais na forma

Pedir ao serviço no `OnInitializedAsync`, mostrar “a carregar” enquanto não há dados, depois a tabela.

O HomeComponent é partilhado, o texto não

Cada app passa a sua mensagem por parâmetro: é assim que se reutiliza sem ficar tudo igual.

Ligar tudo: menu e registo dos serviços

A Entradas no menu

```
Shared/NavMenu.razor, uma entrada por utilidade  
<div class="nav-item px-3">  
  <NavLink class="nav-link" href="temperaturas">  
    Temperaturas  
  </NavLink>  
</div>
```

O href tem de coincidir com o @page do componente. Sem isso, a página existe mas não há forma de lá chegar.

B Registo dos serviços

```
nos dois arranques, para os quatro serviços  
builder.Services  
    .AddScoped<ITemperaturaService,  
        TemperaturaService>();
```

Sem registo, a página rebenta ao abrir com “no registered service of type ...”.

Ordem sugerida para não andar às voltas

1. DTO, interface e serviço de uma utilidade. 2. O componente dessa utilidade. 3. O registo nos dois arranques. 4. A entrada no menu. 5. Compilar e ver no browser. Só depois passar à utilidade seguinte.

Escrever uma vez, correr em vários sítios

Componentes

O .razor não sabe onde corre; o anfitrião decide (Server, WebAssembly, Hybrid).

Interatividade global

Os componentes ficam sem @rendermode e servem Web e MAUI.

RCL

Tudo o que é comum vive numa biblioteca usada pelos dois projetos.

DTO

Os dados que viajam entre partes da app. Na ficha 6 ligam o frontend à API.

Injeção de dependências

Os componentes pedem contratos; o contentor entrega a implementação.

Tempo de vida

Singleton, Scoped, Transient. No servidor, Singleton é de todos.